



Rust

Cristiano Damiani Vasconcellos
cristiano.vasconcellos@udesc.br

Iterator

Iterator é um padrão de projeto que permite percorrer uma coleção de objetos sem a necessidade de expor a representação da coleção. Em Rust o comportamento de um iterator é definido pela trait:

```
pub trait Iterator {  
    type Item;  
    // Required method  
    fn next(&mut self) -> Option<Self::Item>;  
    ...  
}
```

Iterator

```
struct Fib {  
    atual: u128,  
    prox: u128  
}
```

```
impl Fib {  
    pub fn new () -> Self {  
        Self {atual:0, prox:1}  
    }  
}
```

Iterator

```
pub trait Iterator {  
    type Item;  
    // Required method  
    fn next(&mut self) -> Option<Self::Item>;  
    ...  
}  
  
impl Iterator for Fib {  
    type Item = u128;  
    fn next(&mut self) -> Option<u128> {  
        let aux = self.atual;  
        self.atual = self.prox;  
        self.prox = aux + self.atual;  
        Some(aux)  
    }  
}
```

Iterator

```
fn main() {  
    let mut f1 = Fib::new();  
    let f2 = Fib::new();  
  
    for _i in 1..10 {  
        println!("{}", f1.next().unwrap());  
    }  
    println!("{}", f2.map(|x| x * 1000).nth(8).unwrap());  
    let f3 = Fib::new().take_while(|x| x < &1000u128);  
    for i in f3 {  
        println!("{}", i);  
    }  
    let v:Vec<u128> = Fib::new()  
        .take_while(|x| x < &1000u128).collect();  
    println!("{}", v:?);  
}
```

Iterando sobre Listas

```
use std::mem::replace;
```

```
#[derive (Debug)]  
struct NoLista {  
    elem : i32,  
    prox: Option<Box<NoLista>>  
}
```

```
#[derive (Debug)]  
struct Lista {  
    cab : Option<Box<NoLista>>  
}
```

Iterando sobre Listas

```
impl Lista {
  fn new() -> Self {
    Lista {cab: None}
  }
  fn insert (&mut self, v:i32) {
    let n = Box::new(NoLista{elem:v,
                          prox:replace(&mut self.cab, None)});
    self.cab = Some (n);
  }

  fn iter_lista(&self) -> IterLista {
    IterLista::new(&self.cab)
  }
}
```

Iterando sobre Listas

```
struct IterLista<'a> {  
    prox: &'a Option<Box<NoLista>>  
}  
  
impl<'a> IterLista<'a> {  
    fn new(c:&'a Option<Box<NoLista>>) -> Self {  
        IterLista {prox:c}  
    }  
}
```

Iterando sobre Listas

```
impl Iterator for IterLista<'_> {  
    type Item = i32;  
    fn next (&mut self) -> Option<i32> {  
        match self.prox {  
            None => None,  
            Some (p) => {  
                let e = p.elem;  
                self.prox = &p.prox;  
                Some(e)  
            }  
        }  
    }  
}
```

Iterando sobre Listas

```
fn main() {  
    let mut l1:Lista = Lista::new();  
  
    println!("{l1:?}");  
    l1.insert(4);  
    l1.insert(3);  
    l1.insert(2);  
    l1.insert(1);  
    for i in l1.iter_lista() {  
        println!("{i}");  
    }  
    let v1:Vec<i32> = l1.iter_lista().map(|x|x*4).collect();  
    println!("{v1:?}");  
}
```